

Inheritance

Study Guide

Mr. Shivkumar Lilhare
Assistant Professor(CSE), PIT
Parul University

1.	Basics of Inheritance and Access Control.....	1
2.	The super Keyword	5
3.	Summary of Key Concepts.....	7
4.	References.....	8

6.1.1 Basics of Inheritance and Access Control

1. What is Inheritance in Java?

- **Definition:** Inheritance is a fundamental OOP principle where one class (**subclass/child**) inherits properties and behaviors (fields and methods) from another class (**superclass/parent**).
- **Purpose:** Promotes **code reusability**, **hierarchical classification**, and **extensibility**.

2. Benefits of Inheritance

- Reusability of existing code.
- Easier code maintenance.
- Establishes relationships among classes.
- Promotes method overriding and polymorphism.

3. Access Control and Member Access in Inheritance

Modifier	Same Class	Same Package	Subclass	Other Packages
private	✓	✗	✗	✗
default	✓	✓	✓	✗
protected	✓	✓	✓	✓ (if subclass)
public	✓	✓	✓	✓

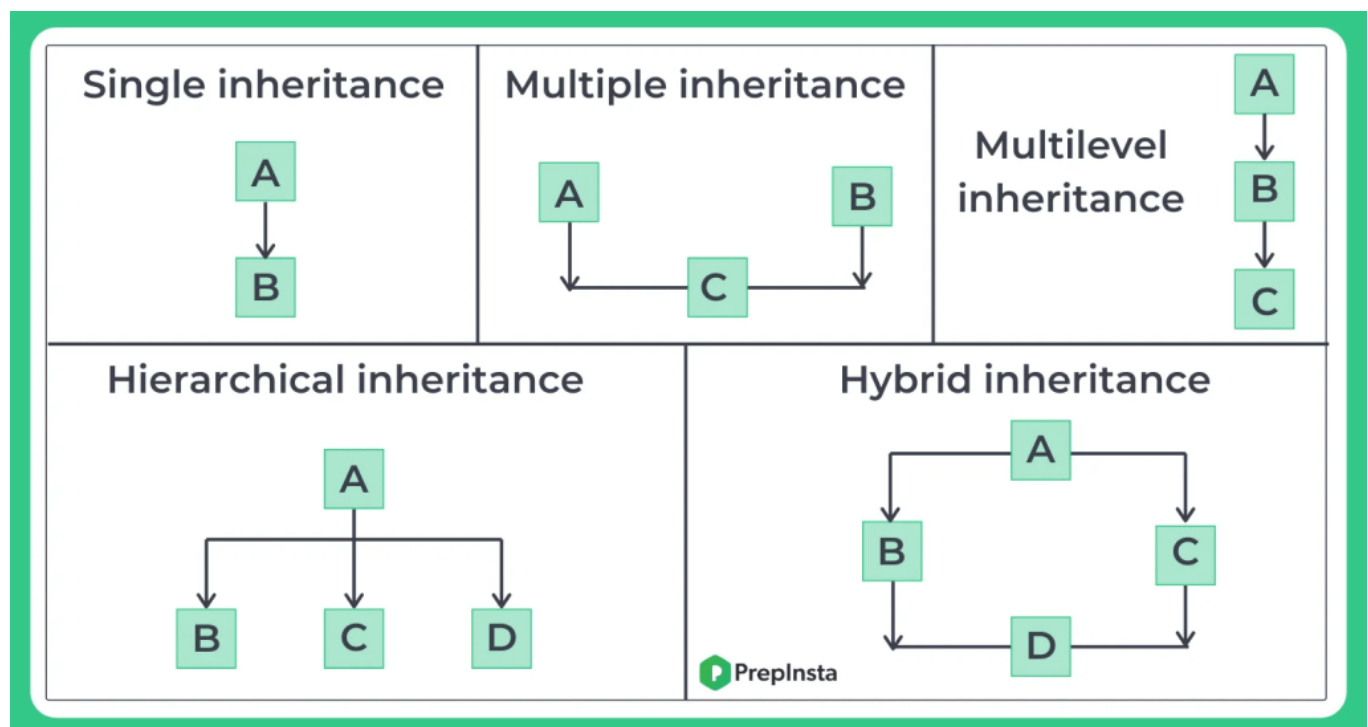
NOTE:

- **Private members:** Not inherited.
- **Protected & public:** Accessible depending on the subclass/package.
- Default members are accessible within the same package.

Forms of Inheritance Supported in Java

Type of Inheritance	Description	Supported in Java?
Single Inheritance	One subclass inherits from one superclass.	✓ Yes
Multilevel Inheritance	A class inherits from a class, which in turn inherits from another class.	✓ Yes
Hierarchical Inheritance	Multiple subclasses inherit from one superclass.	✓ Yes
Multiple Inheritance	A class inherits from more than one class.	✗ Not via classes (Java avoids ambiguity)
Hybrid Inheritance	Combination of multiple and multilevel inheritance.	✗ Not directly supported

Note: Java supports multiple inheritance via interfaces.



Types of Inheritance

Source: <https://studywholenight.com/wp-content/uploads/2023/03/image-1024x546.png>

✓ 1. Single Inheritance

Definition: One class inherits from another.

```
class Animal {
    void eat() {
        System.out.println("This animal eats food");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("The dog barks");
    }
}

public class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat(); // from Animal
        d.bark(); // from Dog
    }
}
```

✓ 2. Multilevel Inheritance

Definition: A class inherits from a derived class, forming a chain.

```
class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}

class Puppy extends Dog {
    void weep() {
        System.out.println("Puppy weeps");
    }
}
```

```
public class Test {  
    public static void main(String[] args) {  
        Puppy p = new Puppy();  
        p.eat(); // Animal  
        p.bark(); // Dog  
        p.weep(); // Puppy  
    }  
}
```

✓ 3. Hierarchical Inheritance

Definition: Multiple classes inherit from a single parent class.

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}
```

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}
```

```
class Cat extends Animal {  
    void meow() {  
        System.out.println("Cat meows");  
    }  
}
```

```
public class Test {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        Cat c = new Cat();  
        d.eat();  
        c.eat();  
    }  
}
```

✗ 4. Multiple Inheritance (Not Supported via Classes)

Java doesn't allow multiple inheritance using classes to avoid **ambiguity**. However, it is allowed using **interfaces**.

```
interface Printable {
    void print();
}

interface Showable {
    void show();
}

class A implements Printable, Showable {
    public void print() {
        System.out.println("Print");
    }

    public void show() {
        System.out.println("Show");
    }
}
```

6.1.2 The super Keyword

- Refers to the **parent class** (superclass).
- Common uses:
 - `super.variableName` – Access parent class variable.
 - `super.methodName()` – Call parent class method.
 - `super()` – Call parent class constructor (must be the first statement in constructor).

✓ 1. Accessing Parent Class Variable

```
class Animal {
    String type = "Animal";
}

class Dog extends Animal {
    String type = "Dog";

    void displayType() {
        System.out.println("Child type: " + type);
        System.out.println("Parent type: " + super.type); // Access parent class variable
    }
}
```

```
public class TestSuperVariable {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.displayType();  
    }  
}
```

Output:

Child type: Dog
Parent type: Animal

✓ 2. Calling Parent Class Method

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        super.sound(); // Call parent class method  
        System.out.println("Dog barks");  
    }  
}  
  
public class TestSuperMethod {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.sound();  
    }  
}
```

Output:

Animal makes a sound
Dog barks

✓ 3. Calling Parent Class Constructor

```
class Animal {  
    Animal() {  
        System.out.println("Animal constructor called");  
    }  
}
```



```
class Dog extends Animal {
    Dog() {
        super(); // Call parent class constructor
        System.out.println("Dog constructor called");
    }
}

public class TestSuperConstructor {
    public static void main(String[] args) {
        Dog d = new Dog();
    }
}
```

Output:

```
Animal constructor called
Dog constructor called
```

➤ **Summary of Key Concepts:**

- Inheritance enables one class (child/subclass) to inherit fields and methods from another class (parent/superclass).
- Benefits: promotes code reuse, supports method overriding, and simplifies code maintenance.
- Access Control in inheritance is determined by access modifiers (private, protected, public, default).
- The super keyword is used to call the superclass's constructor, methods, or variables.
- Forms of Inheritance in Java:
 - Single Inheritance – one subclass extends one superclass.
 - Multilevel Inheritance – a class inherits from another derived class.
 - Hierarchical Inheritance – multiple classes inherit from one superclass.
 - Note: Multiple inheritance is not supported through classes (only via interfaces).

➤ References:

1. • Oracle Java Documentation – Inheritance
 <https://docs.oracle.com/javase/tutorial/java/landl/subclasses.html>
2. • GeeksforGeeks – Inheritance in Java
 <https://www.geeksforgeeks.org/inheritance-in-java/>
3. • Baeldung – Java Inheritance
 <https://www.baeldung.com/java-inheritance>
4. • W3Schools – Java Inheritance
 https://www.w3schools.com/java/java_inheritance.asp
5. • JavaTPoint – super Keyword and Access Modifiers
 <https://www.javatpoint.com/super-keyword>
 <https://www.javatpoint.com/access-modifiers>

Parul[®] University | **NAAC** **A++**
Vadodara, Gujarat | GRADE



    
<https://paruluniversity.ac.in/>